BOUBLIGHA ABDERRAHMAN

# TP : méthodes d’optimisation avec NumPy

## Introduction

Dans ce TP, je compare sept méthodes pour réduire une erreur : GD, GD avec recherche du pas, SGD, Momentum, AdaGrad, RMSprop et Adam. Je les applique à une régression, une classification et un clustering. Le code des modèles et des mises à jour utilise NumPy. Les résultats ci-dessous proviennent des trois jeux disponibles dans l’archive.

## Données utilisées

Les données proviennent de optimisation_datasets.zip, dont le README indique une graine de génération 2026. Les partitions des CSV sont conservées. Pour la régression et la classification, les moyennes et écarts types sont calculés sur le train seulement. La cible de régression est aussi standardisée, puis les prédictions reviennent à l’unité d’origine. Pour Blobs, toutes les observations sont standardisées sans utiliser leurs classes.

## Protocole commun

Les graines sont 42, 123 et 2024. Chaque méthode dispose de 150 époques. GD utilise toutes les données à chaque mise à jour. SGD, Momentum, AdaGrad, RMSprop et Adam utilisent des lots mélangés de 32. Les initialisations sont identiques entre méthodes pour une même graine. On conserve l’état ayant le meilleur critère de sélection, sans modifier le test.

## Choix des paramètres

Deux pas sont comparés avec la graine 42, puis le meilleur est utilisé pour les trois graines. Le critère est la RMSE de validation en régression, la perte de validation en classification et la perte sans étiquette en clustering. Les mémoires commencent à zéro ; mu = rho = beta1 = 0,9, beta2 = 0,999 et epsilon = 10^-8.

## Limite du travail

L’énoncé demande huit jeux de données. Diabetes, Iris, Wine, MNIST et Fashion-MNIST sont absents de l’archive et ne sont pas évalués ici. Ce rapport est donc une synthèse des trois jeux disponibles, et non une validation de tout le sujet. Les tableaux complets et les figures plus grandes se trouvent dans le dossier resultats.

Tableau 1 - Effectifs

| Jeu | Variables | Train | Validation | Test |
|---|---|---|---|---|
| Nonlinear | 2 | 216 | 72 | 72 |
| Spiral3 | 2 | 270 | 90 | 90 |
| Blobs3 | 2 | 360 sans étiquette | - | Même ensemble |

Tableau 2 - Idée simple des sept méthodes

| Méthode | Principe |
|---|---|
| GD | Un gradient sur toutes les données |
| GD-recherche | Un pas calculé ou recherché à chaque mise à jour |
| SGD | Des petits lots mélangés à chaque époque |
| Momentum | Une mémoire des gradients pour guider la descente |
| AdaGrad | Un pas adapté aux gradients accumulés |
| RMSprop | Une moyenne récente des gradients au carré |
| Adam | Deux moyennes et une correction du biais |

# Exercice 1 : régression

## Modèles et gradient

OLS et ridge utilisent une droite dans les deux variables : prediction = w0 + w1*x1 + w2*x2. L'objectif vaut ||Xw-y||²/(2n) + lambda*w^TPw/2. Son gradient est X^T(Xw-y)/n + lambda*Pw, avec P = diag(0,1,1) : le biais n'est pas pénalisé. OLS prend lambda = 0.

## Noyau RBF et recherche du pas

Le noyau Kij = exp(-||xi-xj||²/(2*sigma²)) permet une relation non linéaire. Le gradient est K(Ka-y)/n + lambda*Ka. En mini-lot, seul le terme d'erreur est limité au lot ; la régularisation reste entière. Pour un objectif quadratique de Hessienne H, on minimise l'expression selon eta et on obtient eta = (g^Tg)/(g^THg), lorsque le dénominateur est positif.

## Ce que je remarque

Le tableau 3 montre que le choix de la méthode compte aussi pour le noyau RBF. Adam est retenu sur la validation. Sa RMSE test vaut 0.180 ± 0.004, contre environ 0,82 pour OLS et ridge. Le noyau représente mieux la forme non linéaire des données. Les paramètres retenus sont lambda = 0.001 et sigma = 0.5. La figure 1 compare les pertes ; la figure 2 montre que les prédictions suivent la diagonale et que les résidus sont souvent proches de zéro.

Tableau 3 - RBF, test : moyennes sur trois graines

| Méthode | MSE | RMSE ± é.t. | MAE | R² |
|---|---|---|---|---|
| GD | 0.327 | 0.572 ± 0.000 | 0.490 | 0.517 |
| GD-recherche | 0.049 | 0.221 ± 0.000 | 0.158 | 0.928 |
| SGD | 0.108 | 0.328 ± 0.000 | 0.251 | 0.841 |
| Momentum | 0.050 | 0.224 ± 0.000 | 0.161 | 0.926 |
| AdaGrad | 0.081 | 0.285 ± 0.002 | 0.212 | 0.880 |
| RMSprop | 0.037 | 0.191 ± 0.006 | 0.145 | 0.946 |
| Adam | 0.032 | 0.180 ± 0.004 | 0.137 | 0.952 |

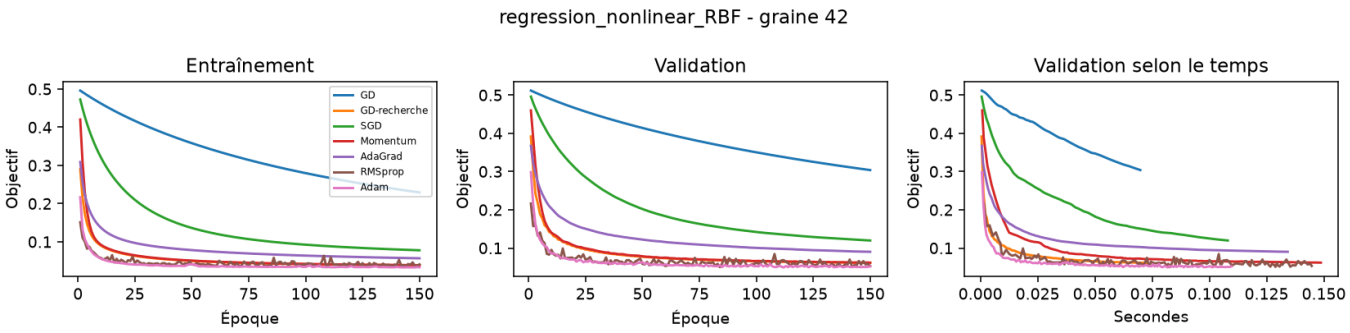



Figure 1 - Pertes selon les époques et le temps, sept méthodes, graine 42.

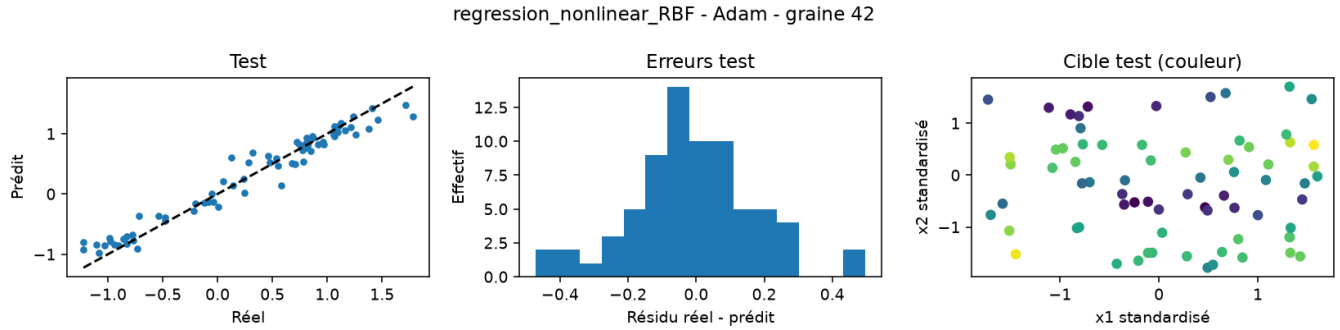



Figure 2 - Prédictions, résidus et cible sur le test ; méthode sélectionnée sur validation, graine 42.

# Exercice 2 : classification des spirales

## Réseau simple

Le réseau possède trois couches entraînables : 2 -> 32 -> 16 -> 3. Les deux couches cachées utilisent tanh. La dernière couche utilise softmax, stabilisé en soustrayant le maximum de chaque ligne. Les poids suivent une initialisation de type Xavier et les biais sont nuls. Chaque classe contient 150 observations, avec 90/30/30 pour train/validation/test.

## Rétropropagation

La perte est l'entropie croisée plus lambda/2 fois la somme des carrés des poids, avec lambda = 0,0001. On part de D3 = (P-Y)/taille_du_lot. Puis D2 = (D3 W3^T) * (1-H2²) et D1 = (D2 W2^T) * (1-H1²). Les gradients des poids sont les produits des activations transposées avec ces erreurs, plus lambda*W. Les gradients des biais sont les sommes des erreurs.

## Ce que je remarque

RMSprop est retenu sur la perte de validation. Son exactitude test moyenne est de 100.00 %. GD progresse moins avec le pas essayé. Le tableau 4 compare aussi la précision, le rappel et le F1 macro. Ces mesures donnent le même poids aux trois classes. La figure 3 montre la baisse de la perte et la figure 4 montre les erreurs et la frontière obtenue. Les résultats restent propres à ce petit jeu de spirales.

Tableau 4 - Test, moyennes sur 3 graines ; écarts types dans resume.csv

| Méthode | Acc. | Préc. | Rappel | F1 | Entropie |
|---|---|---|---|---|---|
| GD | 0.737 | 0.738 | 0.737 | 0.737 | 0.515 |
| GD-recherche | 0.993 | 0.993 | 0.993 | 0.993 | 0.018 |
| SGD | 0.996 | 0.996 | 0.996 | 0.996 | 0.019 |
| Momentum | 0.989 | 0.989 | 0.989 | 0.989 | 0.023 |
| AdaGrad | 0.907 | 0.909 | 0.907 | 0.907 | 0.315 |
| RMSprop | 1.000 | 1.000 | 1.000 | 1.000 | 0.003 |
| Adam | 0.996 | 0.996 | 0.996 | 0.996 | 0.011 |

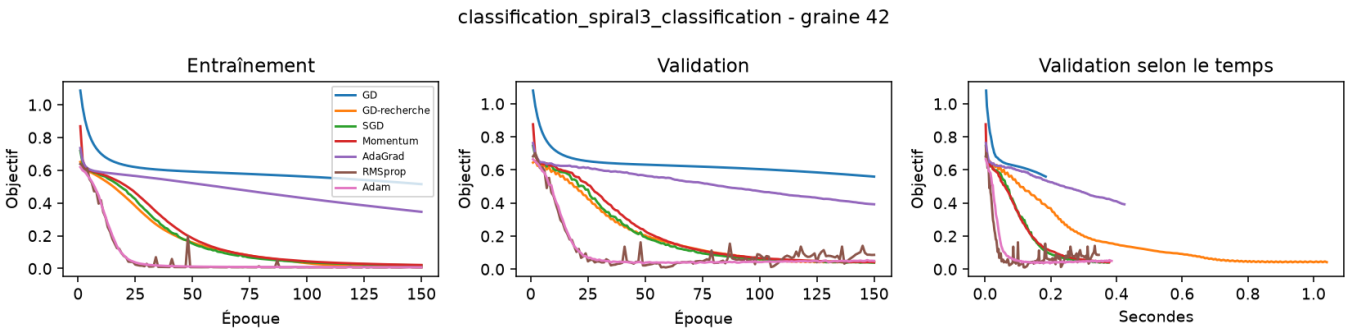



Figure 3 - Pertes selon les époques et le temps, sept méthodes, graine 42.

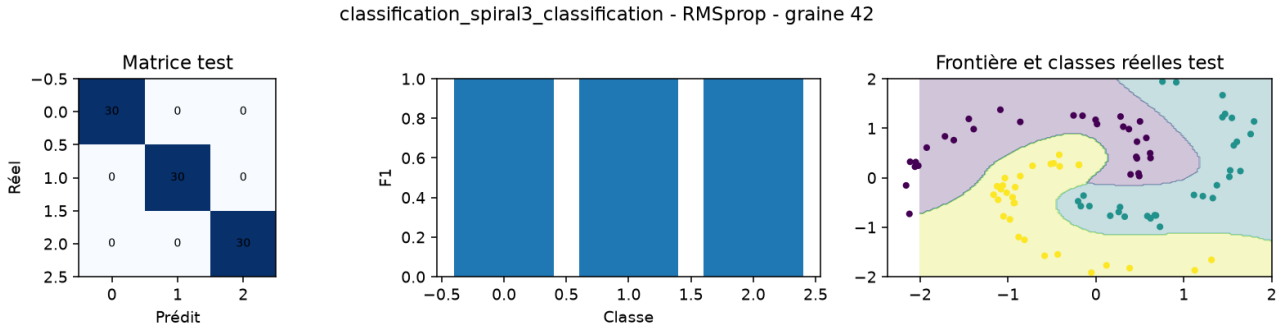



Figure 4 - Matrice de confusion test, F1 par classe et frontière ; graine 42.

# Exercice 3 : clustering des points

## Apprendre sans les classes

On cherche K = 3 centroïdes, initialisés avec trois points distincts. Les étiquettes true_label ne servent ni au gradient ni à la sélection. Elles sont lues seulement pour l'évaluation finale. Tau = 0,25. On utilise les distances D = ||x||² + ||c||² - 2*x^Tc pour éviter de créer un gros tableau de différences.

## Objectif et gradient

L'objectif doux est J = -tau/n * somme log[(1/K)*somme exp(-D/tau)]. Les probabilités Q sont un softmax de -D/tau. Le gradient vaut 2/n * [diag(Q^T1)C - Q^TX]. Le logarithme de la somme des exponentielles est calculé après soustraction du maximum. À la fin, chaque point est affecté au centroïde le plus proche.

## Ce que je remarque

GD avec recherche du pas est retenu selon J sans étiquette. Les sept méthodes donnent des groupes proches, avec un ARI moyen de 0.975. Lloyd sert de référence séparée. Le tableau 5 montre que la faible inertie s'accompagne ici d'une bonne correspondance avec les classes. Ce n'est pas garanti pour d'autres données. Les figures 5 et 6 montrent la descente de l'objectif et les groupes. L'évaluation porte sur les mêmes 360 points que l'apprentissage, sans jeu de test séparé.

Tableau 5 - Moyennes sur 3 graines ; écarts types dans resume.csv

| Méthode | J doux | Inertie | Silhouette | ARI | NMI |
|---|---|---|---|---|---|
| GD | 0.564 | 0.290 | 0.679 | 0.975 | 0.960 |
| GD-recherche | 0.564 | 0.290 | 0.679 | 0.975 | 0.960 |
| SGD | 0.564 | 0.290 | 0.679 | 0.975 | 0.960 |
| Momentum | 0.564 | 0.290 | 0.679 | 0.975 | 0.960 |
| AdaGrad | 0.564 | 0.290 | 0.679 | 0.975 | 0.960 |
| RMSprop | 0.564 | 0.290 | 0.679 | 0.975 | 0.960 |
| Adam | 0.564 | 0.290 | 0.679 | 0.975 | 0.960 |
| Lloyd | 0.564 | 0.290 | 0.679 | 0.975 | 0.960 |

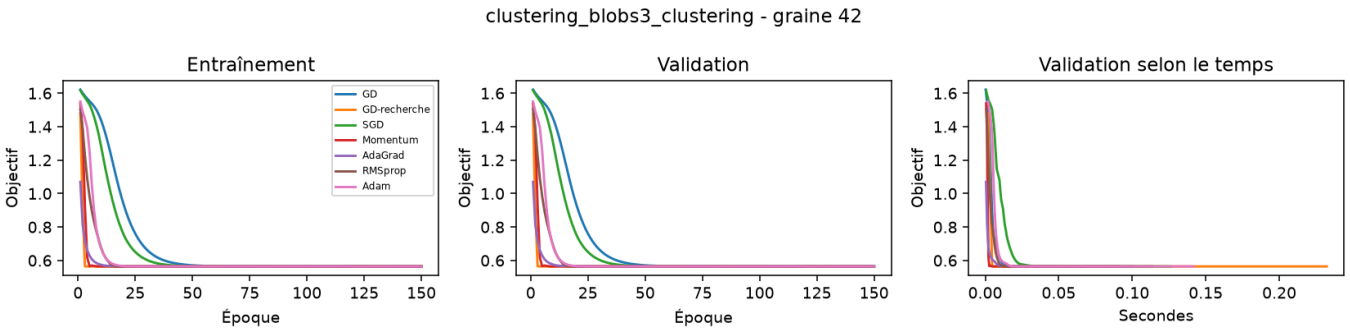



Figure 5 - Pertes selon les époques et le temps, sept méthodes, graine 42.

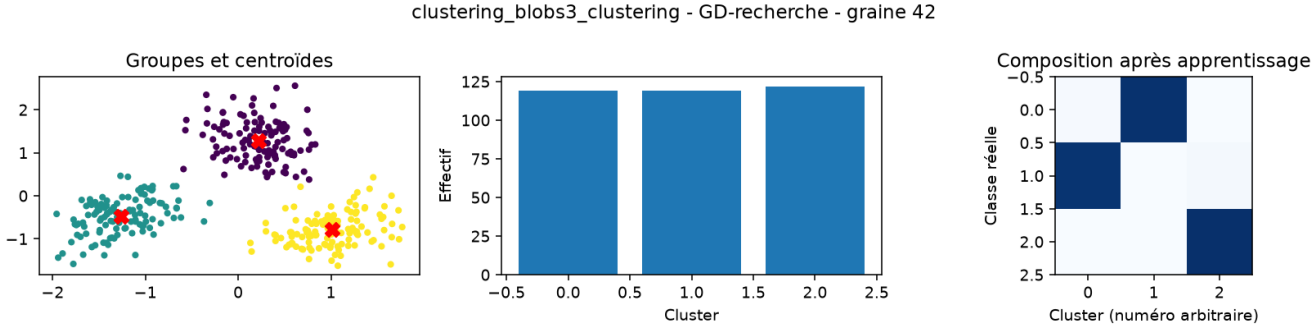



Figure 6 - Groupes, centroïdes, tailles et composition ; graine 42. Les numéros de clusters sont arbitraires.

# Vérifications et conclusion

## Vérification numérique des gradients

Quelques coordonnées de chaque gradient sont comparées avec les différences finies centrales : [J(theta+h)-J(theta-h)]/(2h), pour h = 10^-5. L'erreur relative est |g_num-g| / max(10^-8, |g_num|+|g|). Le maximum observé sur les cinq modèles est 1.96e-09. Pour le noyau, le contrôle porte également sur un mini-lot avec la régularisation complète.

## Coût de calcul

Le tableau 6 présente les coûts moyens des méthodes retenues. Une époque GD fait une mise à jour ; une époque avec lots fait plusieurs mises à jour. Les temps incluent les évaluations de suivi, mais pas les essais de sélection ni le tracé des figures. La recherche numérique effectue plusieurs appels de perte par gradient. L'égalité du nombre d'époques ne signifie donc pas l'égalité du coût. Les courbes complètes selon époques et temps sont enregistrées dans resultats.

## Contrôle et reproduction

Pour OLS, le contrôle utilise une résolution par moindres carrés. Pour ridge, on résout H*w = X^Ty/n ; pour RBF, (K+n*lambda*I)*a = y. Ces solutions contrôlent et sélectionnent les hyperparamètres, mais ne remplacent pas les sept entraînements. L'écart entre les prédictions entraînées et celles du contrôle est dans resultats_bruts.csv. Pour reproduire : installer requirements.txt puis lancer python tp.py --epochs 150 dans le dossier du projet.

## Bilan personnel

Sur ces essais, le noyau RBF décrit mieux la régression non linéaire. Le réseau distingue bien les trois spirales avec plusieurs optimiseurs. Pour Blobs, les méthodes atteignent des groupes similaires. Le choix doit tenir compte de la validation, du temps et de la stabilité entre graines. Les résultats restent limités aux trois jeux fournis et à une petite grille de paramètres. Certaines exigences détaillées de l'énoncé, notamment les jeux réels et leurs visualisations, restent hors de cette version simplifiée.

## Reproduire les résultats

Dans le dossier du projet, installer les dépendances avec python -m pip install -r requirements.txt, puis lancer python tp.py --epochs 150. Les tableaux CSV contiennent les métriques et leurs écarts types. Les figures plus grandes, les contrôles des gradients et les versions sont dans resultats. Sources : fiche de TP de l'enseignant et archive fournie. Aucune donnée manquante n'a été remplacée par un résultat inventé.

Tableau 6 - Coût moyen de l'entraînement final

| Modèle | Méthode | Temps (s) | Mises à jour | Gradients | Pertes |
|---|---|---|---|---|---|
| RBF | Adam | 0.13 | 1050 | 1050 | 300 |
| classification | RMSprop | 0.39 | 1350 | 1350 | 300 |
| clustering | GD-recherche | 0.25 | 150 | 150 | 2400 |